

```

In [1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# =====
# 1. Trouver les variables manquantes
# =====

# 1.1- Lecture des données
titanic_survival = pd.read_csv('titanic_survival.csv')
print("--- Structure des premières lignes ---")
print(titanic_survival.head())

# 1.2- Taille, caractéristiques et cible
# Taille : (1309, 12), Cible : Survived, Caractéristiques : Pclass, Sex, Age, et
print(f"\nTaille du dataset : {titanic_survival.shape}")
print(f"Caractéristiques : {titanic_survival.columns.tolist()}")

# 1.3- Compte détaillé pour la colonne 'Age'
age = titanic_survival["Age"]
age_null = pd.isnull(age)
missing_values = age[age_null]
missing_values_count = len(missing_values)
print(f"\nValeurs manquantes dans 'Age' : {missing_values_count}")

# 1.4- Utilisation de isnull().sum() pour 'Cabin'
cabin_missing = titanic_survival['Cabin'].isnull().sum()
print(f"Valeurs manquantes dans 'Cabin' : {cabin_missing}")

# 1.5- Compte global par colonne
print("\nRésumé des valeurs manquantes :")
print(titanic_survival.isnull().sum())

# 1.6- Note : Il est crucial de gérer les données manquantes car les modèles de
# ne peuvent pas traiter les NaN et cela fausserait les calculs statistiques.

# =====
# 2 & 4.3. Gestion des données manquantes et catégoriques
# =====

# Suppression des lignes sans 'Embarked' (très peu de lignes)
titanic_survival.dropna(subset=['Embarked'], inplace=True)

# Suppression de la colonne 'Cabin' (trop de données manquantes)
titanic_survival.drop('Cabin', axis=1, inplace=True)

# Imputation de l'âge par la moyenne
titanic_survival['Age'] = titanic_survival['Age'].fillna(titanic_survival['Age'].mean())

# Encodage de la variable 'Sex' (Passage en numérique)
titanic_survival['Sex'] = titanic_survival['Sex'].map({'female': 0, 'male': 1})

# =====
# 4. Sélection des caractéristiques (Feature Selection)
# =====

# 4.1- Sélection des colonnes demandées

```

```
colonnes = ['Pclass', 'Sex', 'Age', 'Fare', 'Survived']
titanic_selected = titanic_survival[colonnes].copy()

# 4.2- Note théorique : On identifie Les caractéristiques via la matrice de corr
# L'importance des variables (Random Forest) ou des tests statistiques (Chi-deux

# =====
# 5. Division de L'ensemble de données
# =====

# 5.1- Séparation Features (X) et Target (y)
X = titanic_selected.drop('Survived', axis=1)
y = titanic_selected['Survived']

# Division 80% entraînement / 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random

# 5.2- Affichage des tailles
print(f"\nX_train: {X_train.shape}, X_test: {X_test.shape}")

# 5.3- Intérêt : Évaluer La capacité du modèle à généraliser sur de nouvelles
# données et éviter Le surapprentissage (overfitting).

# =====
# 6. Feature Scaling
# =====

# 6.1- Normalisation
# Standard Scaler (Moyenne=0, Écart-type=1)
sc = StandardScaler()
X_train_sc = sc.fit_transform(X_train)
X_test_sc = sc.transform(X_test)

# MinMax Scaler (Plage entre 0 et 1)
mms = MinMaxScaler()
X_train_mms = mms.fit_transform(X_train)
X_test_mms = mms.transform(X_test)

# --- Commentaire sur Les différences ---
# Le MinMaxScaler compresse Les données dans un intervalle fixe [0, 1],
# ce qui est utile pour Les algorithmes sensibles à L'échelle (ex: KNN).
# Le StandardScaler ne borne pas Les valeurs mais Les centre,
# ce qui est plus robuste aux valeurs aberrantes (outliers).

print("\nTraitement terminé.")
```

--- Structure des premières lignes ---

	PassengerId	Survived	Pclass	\
0	892	0	3	
1	893	1	3	
2	894	0	2	
3	895	0	3	
4	896	1	3	

	Name	Sex	Age	SibSp	Parch	\
0	Kelly, Mr. James	male	34.5	0	0	
1	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	
2	Myles, Mr. Thomas Francis	male	62.0	0	0	
3	Wirz, Mr. Albert	male	27.0	0	0	
4	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	

	Ticket	Fare	Cabin	Embarked
0	330911	7.8292	NaN	Q
1	363272	7.0000	NaN	S
2	240276	9.6875	NaN	Q
3	315154	8.6625	NaN	S
4	3101298	12.2875	NaN	S

Taille du dataset : (418, 12)

Caractéristiques : ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

Valeurs manquantes dans 'Age' : 86

Valeurs manquantes dans 'Cabin' : 327

Résumé des valeurs manquantes :

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	86
SibSp	0
Parch	0
Ticket	0
Fare	1
Cabin	327
Embarked	0

dtype: int64

X_train: (334, 4), X_test: (84, 4)

Traitement terminé.

Réponses aux Questions Théoriques

1.2- Taille, caractéristiques et cible

- **Taille** : Le dataset contient 1309 lignes et 12 colonnes.
- **Caractéristiques (Features)** : PassengerId, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked.
- **Variable Cible (Target)** : Survived.

1.6- Importance de la gestion des données manquantes

Il est crucial de traiter les valeurs manquantes car la majorité des modèles de Machine Learning ne peuvent pas effectuer de calculs sur des valeurs **NaN**. Cela évite les erreurs et les biais statistiques.

4.2- Identification des caractéristiques importantes

On identifie les features via :

- **La Matrice de Corrélation** : pour voir le lien avec la cible.
- **Feature Importance** : (ex: Random Forest) pour classer l'impact des variables.

5.3- Intérêt de la division (Train/Test)

Cela permet d'évaluer le modèle sur des données inconnues et d'éviter le **surapprentissage (overfitting)**.

6.1- Commentaire sur le Feature Scaling

- **StandardScaler** : Centre les données (Moyenne 0, Écart-type 1).
- **MinMaxScaler** : Écrase les valeurs entre 0 et 1.
- **Différence** : Le MinMaxScaler est plus sensible aux valeurs aberrantes (outliers) que le StandardScaler.

In []:

In []: